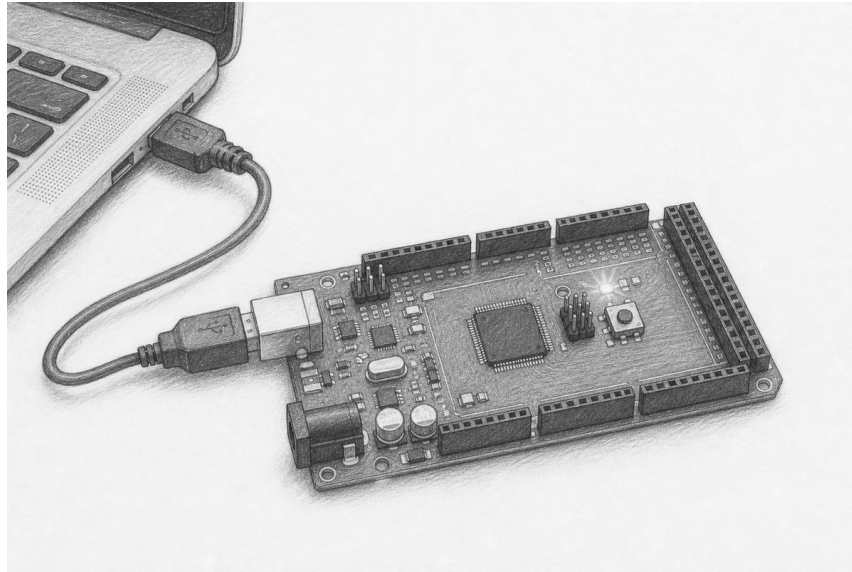


Make state visible

Lesson 001: safe digital output

Predict → claim → command → release



Orientation plate. This graphite view helps you find the Mega 2560, USB connector, and built-in indicator. It is not a pin map. The exact electrical path appears below.

Question	Can an owning C++ object make one hardware state visible, then return its pin to a safe high-impedance state?
Time	50–70 minutes
Energy class	E1: one Mega 2560, its factory LED and resistor, USB power, and the separately identified weak-bias acceptance fixture. Host tests and unpowered inspection are E0.
You need	Arduino Mega 2560, known-good USB data cable, current-limited host USB port, Arch Linux host, <code>arduino-cli</code> , and this ADK checkout
Proof	Independent acquisition status, one short acquisition pulse, three visible behavior cycles, and the host trace proving the controlled high-impedance shutdown command
New words	digital level, output driver, high impedance, ownership, RAI

Safety boundary

Use only the board's factory-installed LED and USB power. Work dry, on a nonconductive surface. Disconnect USB, barrel power, VIN, batteries, shields, and every lead before inspection. Proceed only when the board and cable are undamaged and no header pins touch.

This is an E1 lesson. The teaching circuit is the Mega 2560 factory `LED_BUILTIN` path. The optional acceptance configuration adds only a measured 10 kΩ resistor from D13 to the measured 5 V rail and a DMM from D13 to GND. Install and inspect both with USB removed; record resistor tolerance/rating, meter input resistance/range, and the exact board schematic. No external supply is admitted. USB removal at the computer end is the physical stop. Software shutdown, reset, and an extinguished LED are not emergency stops and do not prove that the pin is high impedance.

Stop immediately for heat, smoke, odor, unexpected power flicker, repeated USB disconnects, or a loose

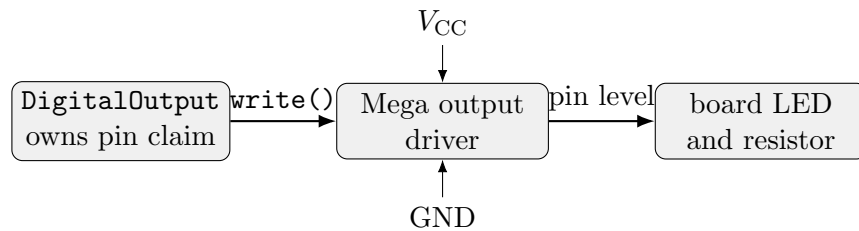
connector. Disconnect upstream power. Do not touch a hot board or retry a damaged one. Label it DO NOT USE.

Check — all power removed before inspection

☐ dry board ☐ clear headers ☐ sound cable ☐ no other power source

1. Separate a command from an electrical state

A digital output actively drives a pin near ground for **Low** or near the board supply for **High**. High impedance, written **HiZ**, means the output driver is disconnected. It is not another voltage level. An unconnected high-impedance pin may float.

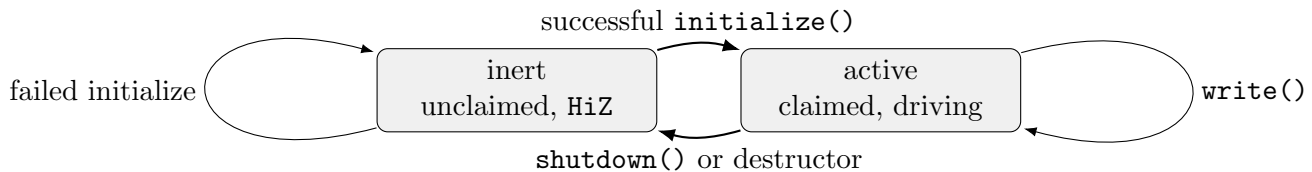


Exact lesson path: software object → LED_BUILTIN output driver → factory LED circuit. No external wire is added.

Predict. Which state actively sources or sinks current? _____ Which state releases the pin? _____

2. Read the ownership contract

ADK treats the pin as a resource, not a global integer. Construction produces an inert object. **initialize()** claims the pin and configures it transactionally. **write()** changes only an initialized, owned output. **shutdown()** is idempotent, selects input mode, and releases the pin as high impedance. Destruction calls shutdown, so stack unwinding in an exception-enabled host still cleans up even though ADK does not throw internally.



Operation	Success	Failure or repetition
construct	inert; no hardware access	not applicable
initialize()	claim, initial level, output mode	invalid or busy; remain inert
write(level)	drive requested level	report status; preserve contract
shutdown()	input mode, HiZ, release	repeated call is harmless
destruct	performs shutdown	never throws

3. Inspect the experimental first-class interface

This interface is host-verified but still experimental; it is not yet a stable supported release. `Runtime` owns the resource registry shared by components. The canonical complete example is `Lesson001DigitalOutput.ino` in its matching `examples` directory.

```
adk::Runtime      runtime;
adk::DigitalOutput output (runtime.resources (), LED_BUILTIN);

adk::Status status = output.initialize ();
if (!status.ok ())
{
    // Stay inert. Report the explicit status.
}

status = output.write (adk::Level::High);
status = output.write (adk::Level::Low);
output.shutdown ();
```

Why no implicit setup? A constructor cannot return a status without exceptions. Explicit initialization makes an invalid pin or resource conflict observable. RAII still guarantees cleanup after successful acquisition.

Why an explicit initial level? Setting the output latch before enabling the output driver avoids a brief unintended pulse. For the board LED, inactive is low. A generic output's safe electrical endpoint is high impedance.

4. Build evidence on the host first

The repository's fake Arduino functions record hardware calls as an ordered trace. Host tests are fast, deterministic, and require no connected board. This excerpt follows the assertions in `tests/test_io.cpp`.

```
fake::reset ();
adk::ResourceRegistry resources;
{
    adk::DigitalOutput output (resources, 13);

    require (output.initialize ().ok (),
             "output initialization");
    require (output.write (adk::Level::High).ok (),
             "output high write");
}
```

The test then checks this exact operation order: write LOW, select OUTPUT, write HIGH, and finally select INPUT during destruction. The claim is then available to a replacement object.

Check — test the boundaries, not only the happy path

☐ initialize success ☐ duplicate pin claim ☐ invalid pin ☐ write before initialize ☐ repeated shutdown ☐ destructor while active

Trace exercise. Write the expected operations when a second output tries to claim an already-owned pin:

5. Compile for the Mega 2560

From the repository root, run the documented checks and the lesson's Arduino compile target. Read the command before executing it. Confirm the fully qualified board name is `arduino:avr:mega`; never infer a connected board or port.

```
make check
make arduino
arduino-cli board list
```

Record the compiler result and reported flash/RAM use:

Host tests

Mega compile

Flash bytes

Static RAM bytes

6. Run the board acceptance procedure

The canonical sketch is a finite, reproducible run. Successful initialization starts a 250 ms acquisition pulse, followed by 750 ms dark separation. Three behavior cycles then alternate 500 ms high and 500 ms low. The sketch calls `shutdown()` after the third low interval and makes no further hardware calls. Pressing reset starts a new run; removing USB leaves the board unpowered.

1. With all power removed, repeat the visual inspection.
2. Connect only the USB data cable.
3. Run `arduino-cli board list`. Copy the Mega's exact port.
4. Upload the lesson sketch using the repository's documented target. Do not paste a port belonging to another device.
5. Predict the acquisition pulse, separation interval, three cycles, and final dark state before upload.
6. Observe the short acquisition pulse separately from the three slower behavior cycles without touching conductors.
7. Wait for the controlled shutdown after the third cycle. Record the LED state, but do not call darkness proof of high impedance.
8. The host trace must end in `INPUT`. For separate physical evidence, install an identified, measured 10 k Ω weak pull-up from TP-D13 to the measured 5 V rail with USB removed. Record the exact board's D13 LED circuit and calculate its effect. Driven LOW must remain within the documented LOW limit; after the final LOW and shutdown, TP-D13 must enter the calculated weak-bias range. If the board circuit or prediction is unresolved, leave this stage blank.
9. Press reset once. Confirm that a new acquisition pulse precedes a new behavior run, then wait for shutdown again.

10. Disconnect USB at the computer end. Confirm that every board indicator is unpowered before handling the board.

Check — bench card — leave incomplete until a person performs it

Date/time: _____ Observer: _____

Board/model/revision or ID: _____

Port and FQBN: _____

ADK revision: _____ `arduino-cli`/AVR core: _____

Supply/source: _____ USB port/cable: _____

USB current limit: _____ measured board supply (if instrumented): _____

Instrument(s), model, and range (or visual only): _____

10 k Ω measured value/tolerance/rating: _____

DMM input resistance and TP-D13 voltage ranges: _____

Exact point: factory `LED_BUILTIN`/D13 path All-power-off inspection: ☐

Factory LED path and weak-bias fixture identified: ☐ board documentation checked: ☐

Initialization status recorded separately: ☐ 250 ms acquisition pulse: ☐

Three 500/500 ms behavior cycles: ☐ controlled shutdown reached: ☐

Host trace ends in `INPUT`: ☐ physical TP-D13 weak-bias result recorded: ☐

reset repeats full sequence: ☐

USB removed; board unpowered: ☐ startup/failure state recorded: ☐ deviation or unexpected behavior: ☐

Predict — acquisition: successful initialization permits one 250 ms pulse before the 750 ms separator.

Observe — acquisition:

Interpret — acquisition:

Predict — behavior: exactly three slower high/low cycles occur only after acquisition evidence.

Observe — behavior:

Interpret — behavior:

Predict — shutdown and reset: after the final driven LOW, `shutdown()` returns D13 to `INPUT` and the identified weak pull-up moves TP-D13 into the calculated range; reset starts at acquisition.

Observe — shutdown, reset, and power removal:

Interpret — shutdown, reset, and power removal:

Deviation, stop action, reviewer, and review date:

Result: ☐ draft ☐ rejected

☐ hardware verified

Reviewer initials: _____

7. Debug with the smallest useful signal

Digital output comes first because one spare pin can expose control flow without a debugger. Assign meanings before running:

Pattern	Meaning
steady inactive	waiting or safely stopped
one short pulse	update loop reached
two short pulses	input event accepted
slow repeated pulse	recoverable status
fast repeated pulse	stop and inspect

Patterns are protocol, not decoration. Keep them deterministic, document the time base, and test the trace. Never use delay-based debug output in code whose timing contract forbids blocking.

8. Deliberately test misuse

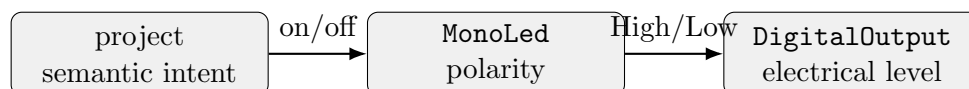
Run these on the host fake. Do not create shorts or faults on hardware.

1. Construct two outputs for one pin. The second initialization must report a conflict without disturbing the first.
2. Call `write()` before initialization. Record its status and confirm no platform operation occurred.
3. Initialize with `NUM_DIGITAL_PINS`. Confirm `error() == StatusCode::InvalidPin`, an empty trace, and no resource claim.
4. Call `shutdown()` twice. Confirm the second call is harmless.
5. Leave scope while active. Confirm destructor cleanup matches explicit shutdown.

Explain. Why is “the LED went dark” weaker evidence than “the pin became high impedance”?

9. Extension: polarity belongs above the endpoint

`DigitalOutput` owns an electrical endpoint and speaks `High/Low`. A later `MonoLed` composes it and speaks `on/off`, accounting for active-high or active-low wiring. Do not teach a generic pin that `High` always means “on.”



Exit ticket

1. What does the object own after successful initialization?
2. Why is HiZ not equivalent to Low?
3. Which operation must be safe to repeat?
4. How does explicit status reporting coexist with RAII?
5. Name one host-only fault test and its required final state.

Ready for lesson 002 when you can: ☐ predict the trace ☐ explain safe shutdown ☐ identify pin conflict ☐ compile for Mega ☐ separate electrical and semantic state

Further reading. Use the companion HTML API page for exact signatures, source links, current limitations, and searchable test names. Use this PDF as the bench worksheet. Arduino's Mega 2560 documentation supplies authoritative board limits; the ADK safety page supplies the project-wide stop conditions. Links are maintained in the HTML lesson to avoid stale printed URLs.